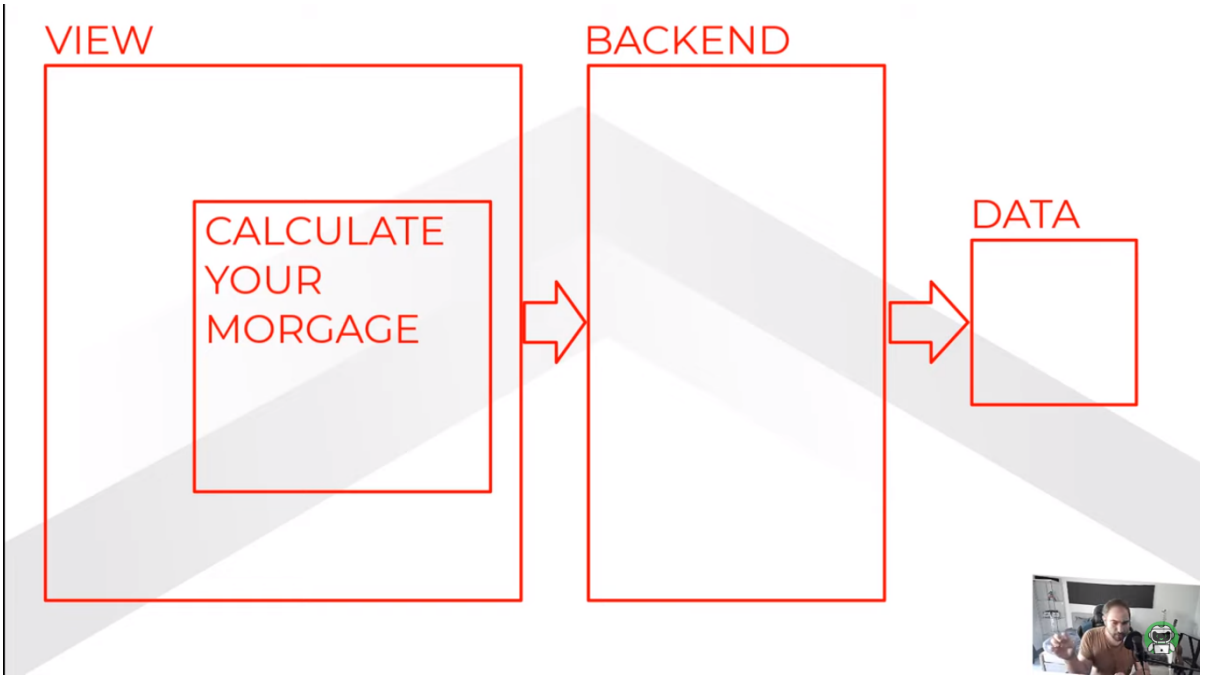
LARAVEL

Livewire

Cuando queremos ejecutar una nueva accion, siempre llamamos una nueva ruta, es decir refrescamos la informacion, para ello se inventaron soluciones, para no tener que refrescar la información, por ejemplo estamos en un sistema para una entidad bancaria y hemos ubicado una calculadora de precios de hipoteca, que no puede utilizarse solo en frontal, si no que va a tener que utilizar el backend para acceder a los datos del historial que tenga el usuario en el banco. la solucion inicial podría ser hacer la calculadora sea en frontal, pero si la usamos utilizando backend (hariamos un formulario que hiciese una peticion a una nueva ruta que calculase el resultado y la devolviese a una nueva vista), en este estaríamos refrescando la información.

Para poder no refrescar y mantenernos en la misma página, acudimos a livewire o Inertia. (renderizado o injección parcial de contenido).

Con el esquema de renderizado parcial, vamos a modificar cual es el paradigma dentro de la arquitectura parcial de los frontales.

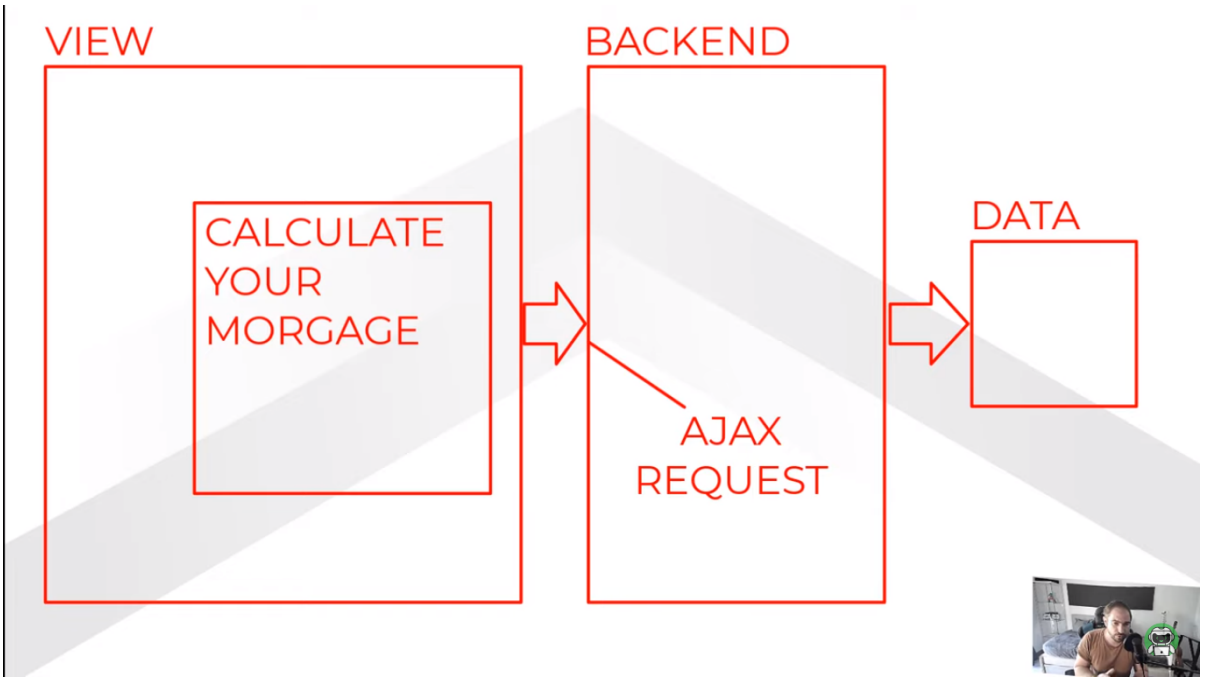


Lo primero que surgió fueron librerias de petición en segundo plano como AJAX, que nos permitia solicitar información, como si fuese una información secundaria como una api, para obtener los resultados y modificar el DOM de la página y cambiar parcialmente el dato.

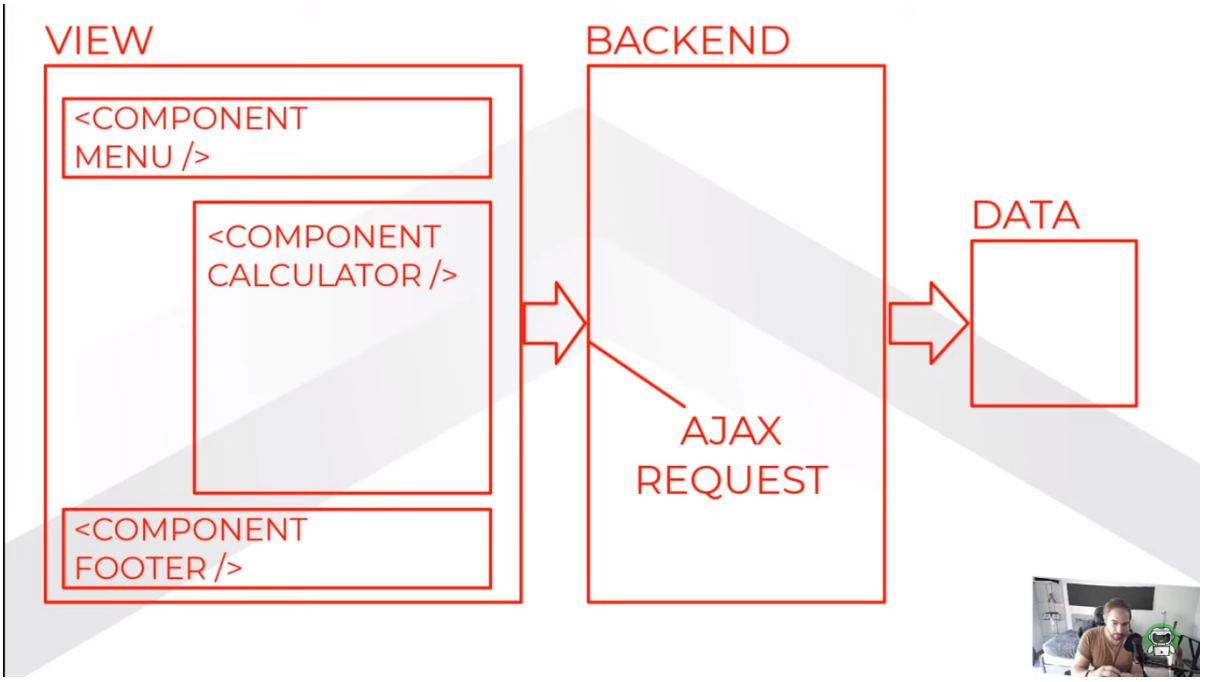
Posteriormente se utilizo JQuery, y poco a poco se fueron buscando mas utilizades para la renderización parcial para la utilización como en las redes sociales, (donde etiquetamos personas, damo me gusta y no queremos que la página vuelva a cargar), donde empezó a hacerse imprescindible la renderizacion.

Se creo el modelo MVM (modelo vista modelo) mediante el cual nos permitía hacer construcciones de frontales que podian hacer el renderizado parcial del contenido, este tipo de librerias se basan en la modificacion de como conocemos los frontales.

Antes los html era contenido que terminabamos lanzando o renderizando en nuestro navegador de usuario, sin embargo ahora nuestro interes no va a ser tanto el de renderizar el html de forma estática (server side rendering),



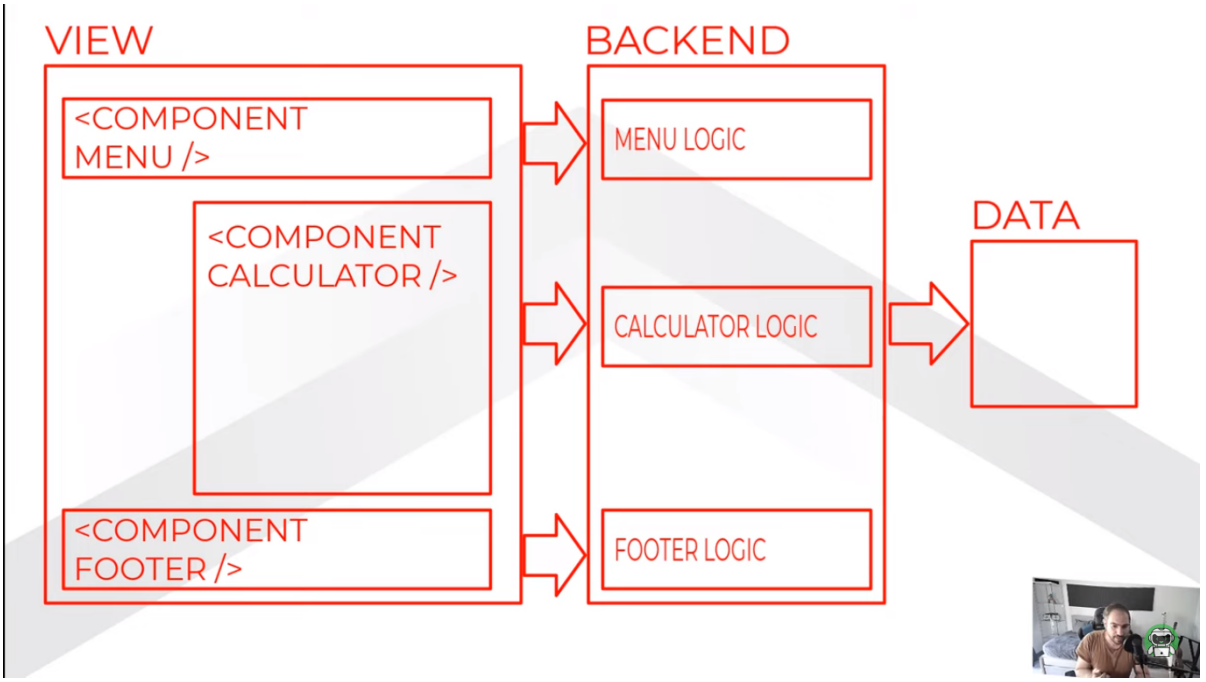
si no lo que vamos es generar una serie de componentes, pensando en como podemos desglosar nuestra página en una unidad mínima que vamos a llamar componente, añadiendo una composición con diferentes componentes, llegamos a la formación de la vista (Por ejemplo los componentes blade que hicimos con las tarjetas), Por ejemplo, podemos tener el componente menu, componente pie, ...



Hay que pensar que las vistas no es una única entidad, si no que va a estar formada por diferentes componentes, por ejemplo pensar esta vista va a tener el componente menu, el componente mensaje flash, componente tarjeta, el componente footer. Aqui es donde se utilizar el MVM (modelo vista modelo) , para entender esto en profundidad es interesante conocer vue, puesto que se vasa en este modelo.

Aqui cada componente va a tener un funcionamiento propio, que es utilizable en numerosas vistas, por ejemplo el componente calculadora, se puede utilizar en numerosas vistas, por tanto tiene la ventaja de no tener que refrescar toda la vista, si no que tambien poder conseguir una mejor utilibilidad de los componentes.

Conseguimos no refrescar, por que cada componente no solo va a contener la vista, si no que tambien va a contener la lógica que necesita para funcionar, es decir si yo genero el componente de la calculadora, éste va a contener la vista de la calculadora (template de la calculadora) y tambien los datos y funciones necesarias para hacer funcionar esa calculadora.



Asi lo que queremos es que la vista no sea estática, si no que lo que queremos son vistas dinámicas en las que va a poder refrescar el contenido sin la necesidad de llamar a otra url diferente, ademas buscamos el beneficio de tener intefaces mas utilizables ya que vamos a encapsular su componente.

Asi cuando instalamos breeze o jetstream vimos que las vistas utilizaban componentes, que empezaban con las etiquetas y que eran las que conformaban las vistas, y estos se almacenaban en la carpeta components.

Por defecto no viene laravel con esta configuración , por que igual que tiene ventajas, tambien tiene desventajas:

- El fomato de renderizacion parcializada, es que nos arrastra fallos en seo (posicionamiento) para la busqueda por que la información no ha sido injectada, por eso hay que hacer una distinción de la funcionalidad que va a tener cada parte de la aplicación. Por ejemplo facebook o instagram, el muro no es interesante para el seo de la página, sera importante para el seo la presentación de la aplicacion de facebook que estará fuera de la propia aplicacion.

Librerias y frameworks reactivos:

- react: desarrollada por facebook
- vue: desarrollada por la comunidad
- angular

- svelte

Estan desarrolladas para trabajar principalmente en javascript, o su superconjunto typescript. olvidando realizar peticiones a AJAX o de JQuery que se necesitaban para hacer ese renderizado parcial.

Si utilizamos laravel para un backend puro que va a estar arrojando una api, este tipo de tecnologias son fantásticas, porque los desarrolladores de backend al no preocuparse de las vistas, este tipo de tecnologías vienen a sumar, seguimos desarrollando el backend como siempre, y con estas tecnologias conseguimos que puedan ser consumidos, otorgando nuevos beneficios al usuario.

Para consolidar el backend con este tipo de tecnologias, surgen tambien las capas por grafos.

Si utilizamos laravel en estructura monolítica o parcialmente monolítica (api y monolito), aqui nos encontramos un problema, por que blade no es reactivo y habría que utilizar AJAX y no es lo que queremos. Para ello se utiliza livewire e inertia, que son las soluciones reactivas para los frontales.

Inertia permite utilizar vue en las vistas de blade (añadir componentes de vue, para ello hay que saber vue) y livewire, permite obtener las mismas funcionalidades reactivas, pero con la sintaxis de blade.

Inertia al utilizar vue hay que saber javascript y en livewire se sigue utilizando php.

Instalar livewire

Nos generamos un nuevo proyecto

Creamos un proyecto nuevo "livewireexample" para hacer los ejemplos de esta sección, hay que tener en cuenta que si vamos a utilizar utilidades propias de autenticación hay que tenerlo en cuenta a la hora de crear el proyecto para no tener problemas al instalar el paquete con posterioridad

```
alias laravel='/var/www/html/curso-laravel/vendor/bin/laravel'

laravel new livewireexample
cd livewireexample
composer update --ignore-platform-reqs
php artisan key:generate
php artisan config:clear
```

1. Creamos la bd: livewireexample
2. Configuramos archivo .env

Instalar livewire

```
composer require livewire/livewire
```

Antes de empezar a trabajar hay que añadir a todas las vistas que utilicen los estilos y scripts de livewire, para ello creamos un layouts/app.blade.php donde los vamos a incluir

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>@yield('title')</title>
    @livewireStyles
</head>
<body>
    @yield('content')
    @livewireScripts
</body>
</html>
```

@livewireStyles: incluye los estilos de livewire

@yield('content'): para incluir el contenido que le llegue

@livewireScripts: incluye los scripts de livewire

@yield('title'): para incluir el titulo que le llegue

Para probar, borramos el contenido de welcome.blade.php y realizamos el siguiente código

La documentación oficial está en : https://laravel-livewire.com/

Livewire utiliza AJAX internamente para hacer el renderizado de los componentes con un nuevo html, mutando al DOM (modelo de objeto de documento , es una interfaz de programación para los documentos HTML y XML) con los nuevos cambios.

1. Creamos un componente de livewire para hacer un contador

   ```
   php artisan make:livewire counter
   ```

   Esto nos crea una carpeta livewire en app con counter.php (Aqui va la lógica de los componentes de livewere) y en resources/views tambien ha creado una carpeta livewire con el archivo counter.blade.php (donde ubicaremos la vista del componente).

2. Vamos a utilizar el contador en la vista de welcome.blade.php los componentes siempre tienen que tener una etiqueta raiz donde incluye todo su contenido

3. En views/livewire/counter.blade.php

   ```
   <div>
       {{-- A good traveler has no fixed plans and is not intent upon arriving.
   --}}
       <h1>Hello from counter</h1>
   </div>
   ```

4. En welcome.blade.php, en lugar de pintar el hello world vamos a incluir el componente, hay dos maneras

1. indicando el livewire

```
@extends('layouts.app')
@section('title', 'Livewire example')

@section('content')
    {{-- opcion 1 --}}
    @livewire('counter')
@endsection
```

2. Hacerlo parecido a vue, marcando una etiqueta

```
<livewire:counter />
```

   Se pueden añadir varios componentes a la vista, cuando la aplicación tiene un solo componente se denomina SPA (Single Page Application)

3. Creamos un contador en counter.blade.php

```
<div>
    {{-- A good traveler has no fixed plans and is not intent upon
arriving. --}}
    <h1>{{ $count }}</h1>
</div>
```

   $count es una variable con el valor que va a pintar la vista

   La diferencia es que ahora a esta vista vamos a poder incluir vinculaciones a funciones que vamos a tener en la lógica, por ejemplo nos creamos un boton que va a ir incrementando el valor del contador, antes en blade lo habríamos hecho con una url con un enlace al que le pasaríamos los valores, ahora ya no, será la lógica la que haga la funcionalidad

```
<div>
    {{-- A good traveler has no fixed plans and is not intent upon
arriving. --}}
    <h1>{{ $count }}</h1>
    <button wire:click="increment">Increment</button>
</div>
```

   Para llamar a la función que va a hacer la funcionalidad, podríamos utilizar el evento onclick, pero lo vamos a hacer con la directiva `wire:click="increment"`

4. En Counter.php siempre tiene una función **render** que es la encargada de retornar la vista (renderización del componente), a diferencia de lo que ocurre con el método return view de la vista de blade, nuestro componente está envolviendo este return view dentro de su función render, que es la que se va a encargar de hacer el migrate (actualización) del contenido en la vista, para que no tengamos que hacer el refresco en pantalla.

```
public function render()
{
    return view('livewire.counter');
}
```

NOTA: cada componente va a tener una función render que es la que retornará la vista (renderización)

5. ,Crear la lógica del componente, en Counter.php declaramos una variable publica (para poderla utilizar en todo el proyecto) que es la misma que se va a mostrar en la vista

```
public $count = 0;

public function increment()
{
    $this->count++;
}
```

Cuando se ejecuta en el navegador, observamos como se ejecuta el componente, sin recargar otra url, recarga el componente.

Pilares fundamentales:

1. wire:click: para vincular funciones (como en el ejemplo anterior)
2. hooks al ciclo de vida
3. data binding bidireccional: que nos permite vincular información entre la parte lógica (modelo) y la parte de la vista.

Hook del ciclo de vida

Cuando cargamos el componente dentro de una vista, este componente va a tener un ciclo de vida dentro de nuestra aplicación, de tal forma que cuando es pintado por primera vez, es decir que esta componente ha sido montado, éste está a la escucha de posibles modificaciones, y cuando el contenido de éste se modifica, decimos que ocurre un evento dentro del ciclo de vida de actualización (este componente ha sido actualizado), por último podemos indicar que éste componente desaparezca de la vista, que en función de las condiciones que tenemos en nuestro contenido, decidamos que se oculte o eliminarlo de nuestra vista, sin haber abandonado nuestra vista. en este momento decimos que el componente ha sido destruido o desmontado. Además de estos momentos existen más, para ello existen funciones a utilizar cuando ocurre un evento (antes o después de la actualización).

Por ejemplo, podríamos querer que cada vez que ocurra un evento de actualización, guardemos un log o hacer acciones paralelas.

mount: cuando el valor es creado

```
public function mount()
{
    // this->count = 20;
    $this->fill(['count' => 20]);
}
```

Esta función se ejecuta cuando el componente es montado.

Da error `this->count por que como no esta montado no existe el count, por tanto lo hacemos asi: $this->fill(['count' => 20]);`

Ahora si ejecutamos el proyecto, cuando inicia, el valor por defecto es 20

updating: cuando se esta ejecutando (se ejecuta antes de actualizar)

updated: cuando es actualizado (se ejecuta después de actualizar)

Data binding bidireccional

Se utilizará sobre todo en componentes para formularios

Por ejemplo, para probar creamos un input en la vista counter.blade.php y declaramos una variable en el controlador

En el componente counter.php, declaramos una variable, para que según vayamos escribiendo el texto en el input se vaya guardando automáticamente en dicha variable.

```
public $username = ""
```

Asi por ejemplo en la función click podemos hacer guardar username y podemos acceder a ésta y saber su contenido.

En la vista counter.blade.php

```
<input type="text" wire:model.live="username"/>
<br />
<h3>{{ $username }}</h3>
```

La sintaxis es parecida a vue, cambiaría que en lugar de wire:model sería vue:model. Con wire:model vinculamos el valor del input con la variable username. Dibujamos el valor de la variable para ver como se va actualizando el valor de la variable según lo que escribamos en el input.

NOTA: en laravel 11 hay que poner wire:model.live="variable" en lugar de wire:model="variable" como en las versiones anteriores.

Ahora hacemos un ejemplo completo de livewire con crud.

1. Creamos un modelo Note con sus migraciones

```
php artisan make:model Note --migration
```

2. Añadimos el guard al modelo Note

```
protected $guard="";
```

3. Contruimos la estructura de la migracion create_notes_table

```
public function up(): void
{
    Schema::create('notes', function (Blueprint $table) {
        $table->id();
        $table->string("content")->nullable();
        $table->timestamps();
    });
}
```

4. Ejecutamos las migraciones

```
php artisan migrate
```

5. Aprovechamos los archivos de vista y logica de counter de livewire para el ejemplo

    1. En counte.blade.php

    ```
    <div>
        {{-- A good traveler has no fixed plans and is not intent upon
    arriving. --}}
        <input type="text" wire:model.live="note" />
        <button wire:click="store">Save Note</button>
    </div>
    ```

    2. En Counter.php declaramos la variable note y creamos la función store que es la que va a guardar el valor de la nota en la bd

    ```
    use App\Models\Note;

    public $note = "";

    public function store()
    {
        Note::create([
            "content" => $this->note
        ]);
    }
    ```

    Nos creamos además la variable feedback para indicarle al usuario que esta sucediendo, por ejemplo indique después de crear la nota, que la nota se ha creado

```
public $feedback ="";

public function store()
{
    Note::create([
        "content" => $this->note
    ]);

    $this->feedback = "Note created";
}
```

Y en la vista añadimos que dibuje el valor de feedback

```
<p style="color: red">{{ $feedback }}</p>
```

6. Podríamos modificar en la función render, que muestre las notas

```
public function render()
{
    $notes = Note::all();
    return view('livewire.counter', compact('notes'));
}
```

Si nos diese error compact podríamos forzarlo `return view('livewire.counter', ['notes' => $notes]);`

Y en la vista añadimos que las muestre

```
@foreach ($notes as $note)
    <p>{{ $note->content }}</p>
@endforeach
```

7. Podríamos añadir a cada nota creada un botón para eliminar dicha nota
    1. En la vista modificamos

```
@foreach ($notes as $note)
        <p>{{ $note->content }} <button wire:click="destroy({{ note->id }})">Destroy</button></p>
    @endforeach
```

    2. En counter.php creamos la función delete

```
public function destroy($id){
    Note::destroy($id);
    $this->feedback = "Note deleted";
}
```

8. Podríamos modificar la nota

1. Modificamos en la vista, para añadir un boton para update (utilizamos el mismo input de crear)

```
<button wire:click="update({{ $note->id }})">Update</button>
```

2. Creamos en counter.php la funcion update

```
public function update($id){
    $noteToUpdate = Note::find($id);
    $noteToUpdate->content = $this->note;
    $noteToUpdate->save();
    $this->feedback = "Note updated";
}
```

Inertia

Inertia se basa en la reactividad a traves de vue. Va a permitir a laravel utilizar vue en frontales, como un monolito moderno. De esta forma si no se conoce vue, puede ser mas complicado entenderlo.

Si sabemos javascript y vue es mejor utilizar Inertia, si no utilizar livewire (ambos son buenas opciones).

Nos generamos un nuevo proyecto

Creamos un proyecto nuevo "inertiaexample" para hacer los ejemplos de esta sección, hay que tener en cuenta que si vamos a utilizar utilidades propias de autenticación para no tener problemas al instalar el paquete con posterioridad

```
alias laravel='/var/www/html/curso-laravel/vendor/bin/laravel'

laravel new inertiaexample
cd inertiaexample
composer update --ignore-platform-reqs
php artisan key:generate
php artisan config:clear
```

1. Creamos la bd: inertiaexample
2. Configuramos archivo .env

Vamos a utilizar también jetstream con inertia

```
composer require laravel/jetstream
php artisan jetstream:install inertia
```

NOTA: si quisiéramos instalar solo inertia sin jetstream ni breeze `composer require laravel/inertia`

Cargamos las migraciones

```
php artisan migrate
```

Si nos vamos a routes/web.php vemos que en lugar de retornar la vista con `return view` lo hace de la siguiente manera: `return Inertia:render`

```
Route::get('/', function () {
    return Inertia::render('Welcome', [
        'canLogin' => Route::has('login'),
        'canRegister' => Route::has('register'),
        'laravelVersion' => Application::VERSION,
        'phpVersion' => PHP_VERSION,
    ]);
});
```

Retorna la vista con Inertia y el método render, enviando la vista y las variables que va a necesitar para funcionar.

Los **componentes de jetstream** están en `Resources/js/Components` y en dicho directorio contenemos por ejemplo el componente web.vue con el codígo escrito en vue.

1. Creamos un controlador

   ```
   php artisan make:controller ExampleController
   ```

2. Creamos una ruta en web.php

   ```
   use App\Http\Controllers\ExampleController;

   Route::get('/example', [ExampleController::class, 'index'])->name('example');
   ```

3. Creamos un componente en vue en Resources/js/components , para ello creamos un archivo ExampleInertia.vue

   Los componentes de vue, van a tener mínimo 3 etiquetas:

   - template: donde ubicamos la vista, el contenido html que va a ser renderizado.
   - script: donde está toda la lógica, los datos, las funciones que vamos a querer utilizar. Vue ofrece varias formatos de trabajo
     - Composition API: Se introdujo para abordar algunas de las limitaciones de la API de opciones. Permite a los desarrolladores utilizar un estilo de programación reactivo y funcional para crear componentes y ofrece una forma más flexible y expresiva de definir el comportamiento de los componentes.
     - Option API: Implica el uso de un conjunto de opciones, como datos, métodos y propiedades calculadas, para definir el comportamiento y el estado de un componente.

        - Composition API en su versión reducida, que es la de script setup que es la que vamos a utilizar
    - style: donde se encuentra el css que queremos aplicar, si le aplicamos `scoped` a los estilos css, quiere decir que estos son utilizados para sólo este componente. Asi evitamos colisión entre diferentes estilados.

```
<template>
  <!-- Código HTML -->
</template>

<script setup>
  // Código Javascript información sobre lógica y datos
</script>

<style scoped>
  /* Código CSS */
</style>
```

Nota: para los que no conocen vue, el nombre de los archivos se hace con (camelCase) , se suelen utilizar dos palabras, mas la extensión .vue.

Para nuestro ejemplo creamos:

```

```

Si al ejecutarlo, muestra el error: `Unable to locate file in Vite manifest: resources/js/Pages/ExampleInertia.vue.` es porque hemos ubicado la página dentro del componente en lugar de en el directorio (pages) páginas. Esto pasa porque a veces vue no diferencia entre lo que es un componente y lo que es una pagina (ambos son lo mismo). Pero Jetstream si hace dicha distinción, por tanto si usamos Inertia con Jetstream (nuestro caso), va ha hacer una distinción entre un componente que es renderizado como una vista general, que va a ser llamado a través del controlador, y un componente que es renderizado dentro del contenido de nuestro proyecto. Para ello, lo que hay que hacer es mover el archivo de resources/js/components a resources/js/pages.

Una vez movido el archivo ejecutar npm run para que se de cuenta del cambio.

Esto es debido porque cuando generamos nuestro contenido con Inertia, lo que hace es que nuestro proyecto utiliza el app.js, que lo que le dice a la configuración de vue (main.ts o main.js) que tiene resolver el contenido del directorio pages

```
resolve: (name) => resolvePageComponent(`./Pages/${name}.vue`,
import.meta.glob('./Pages/**/*.vue')),
```

Asi con `npm run dev` se recargan las páginas.

4. Creamos la función index en ExampleController.php para mostrar la vista, pero en lugar de retornarla con `return view();` lo hacemos a través de un componente vue `return inertia:render('NombreComponente')`

```php
use Inertia\Inertia;

public function index()
{
    return inertia::render('ExampleInertia');
}
```

Dentro de un componente vue, puede recibir prods (información necesaria para funcionar), en el caso de que estemos utilizando esté formato, cuando le pasamos los datos, nuestro componente lo utilizará como prods.

NOTA:

El funcionamiento es el siguiente.

- La ruta llama a el controlador (como siempre)
- El controlador funciona como siempre, pero en lugar de retornar una vista (return view()) retorna un inertia::render. para que el sistema entienda que lo que va a estar renderizando es un componente vue, es decir que es estamos cargando es un .vue y no .blade.php
- En resources en lugar de en views trabaja con javascripts (directorio js)
    - layouts:
    - components:
    - pages: los componentes que vamos a utililzar para vincular directamente a una ruta. Estos componentes pueden ser cargados por que tenemos el app.js (que es el punto de entrada (se encarga de cargar la aplicación de vue)).

Carga todo lo que se encuentre en el directorio de pages y por ello hay que cargarlo, ya que esta en javascript.

Necesitamos el comando `npm run dev` si estamos en entorno de desarrollo o `npm run build` cuando ya vayamos a pasar a producción.

Vue va a estar generando archivos estáticos que son los que dan soporte a todo el funcionamiento.

En este caso como estamos en un entorno dinámico a través de laravel, necesitamos indicar cuando levantamos el servicio, a la parte javascript que tiene que generar sus archivos para que todo funcione

Vamos a hacer un ejemplo de vue (vue puro)

Tenemos que tener en cuenta dos cosas:

1. El paso de información, cuando recepcionamos información dentro de un componente, éste va a ser tratado dentro del componente como una **PROP** (atributos personalizados que podemos pasar a los componentes en Vue. Actúan como parámetros de un componente, permitiendo que datos específicos fluyan desde el componente padre hacia el componente hijo)

2. Como navegamos, no vamos a estar utilizando el enrutador de vue, si no el de laravel, para ello vamos a tener un componente ya preconstruido que es **navlink**

Ejemplo de reactividad

En resources/js/Pages/ExampleInertia.vue, tenemos la referencia que es un dato reactivo que vamos a poder estar utilizando, importamos las referencias de vue

```
<script setup>
    import{ref} from 'vue'

    const counter = ref(0);
    const increment = () => {
        counter.value++
    }
</script>
```

Declaramos una variable contador (counter) y una función de javascript (increment) que sume 1 al contador.

Las variables de se pueden utilizar en el